

```
!pip install transformers datasets torch accelerate sentencepiece gradio
```

Show hidden output

```
import json
```

```
with open("dataset.json", "r") as f:
    data = json.load(f)
```

```
print(data[:2])
```

```
[{'question': 'What is RCOEM?', 'answer': 'RCOEM is Shri Ramdeobaba College of Engineering and Management located in Nagpur.'}]
```

```
inputs = []
targets = []
```

```
for item in data:
    inputs.append("question: " + item["question"])
    targets.append(item["answer"])
```

```
print(inputs[0])
print(targets[0])
```

```
question: What is RCOEM?
```

```
RCOEM is Shri Ramdeobaba College of Engineering and Management located in Nagpur.
```

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
```

```
model_name = "google/flan-t5-base"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
```

```
warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
config.json:      1.40k/? [00:00<00:00, 19.6kB/s]
```

```
tokenizer_config.json:    2.54k/? [00:00<00:00, 34.7kB/s]
```

```
spiece.model: 100% 792k/792k [00:00<00:00, 3.94MB/s]
```

```
tokenizer.json:    2.42M/? [00:00<00:00, 28.9MB/s]
```

```
special_tokens_map.json:  2.20k/? [00:00<00:00, 61.4kB/s]
```

```
model.safetensors: 100% 990M/990M [00:11<00:00, 81.1MB/s]
```

```
Loading weights: 100% 282/282 [00:01<00:00, 364.96it/s, Materializing param=shared.weight]
```

```
The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.weight, but both are present in
generation_config.json: 100% 147/147 [00:00<00:00, 7.08kB/s]
```

```
encodings = tokenizer(
    inputs,
    truncation=True,
    padding=True,
    max_length=128
)
```

```
labels = tokenizer(
    targets,
    truncation=True,
    padding=True,
    max_length=128
)
```

```
encodings["labels"] = labels["input_ids"]
```

```
import torch
```

```
class ChatDataset(torch.utils.data.Dataset):
    def __init__(self, encodings):
```

```

        self.encodings = encodings

    def __getitem__(self, idx):
        return {
            key: torch.tensor(val[idx])
            for key, val in self.encodings.items()
        }

    def __len__(self):
        return len(self.encodings["input_ids"])

dataset = ChatDataset(encodings)

```

```

from transformers import TrainingArguments, Trainer

training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=10,
    per_device_train_batch_size=2,
    logging_steps=1,
    save_steps=5
)

```

```

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset
)

trainer.train()

```

[Show hidden output](#)

```

model.save_pretrained("rcoem-chatbot")
tokenizer.save_pretrained("rcoem-chatbot")

```

Writing model shards: 100% 1/1 [01:16<00:00, 76.70s/it]
('rcoem-chatbot/tokenizer_config.json', 'rcoem-chatbot/tokenizer.json')

```

def ask_question(question):
    input_text = "question: " + question

    inputs = tokenizer(input_text, return_tensors="pt")

    outputs = model.generate(
        **inputs,
        max_length=50
    )

    response = tokenizer.decode(outputs[0], skip_special_tokens=True)

    return response

print(ask_question("Does RCOEM provide hostel facilities?"))

```

RCOEM provides hostel facilities.

```

import gradio as gr
import json

# Load dataset
with open("dataset.json", "r") as f:
    data = json.load(f)

# Chatbot Function
def chatbot(message, history):

    message = message.lower().strip()

    best_match = None
    best_score = 0

    # Search through dataset
    for item in data:

        text = (
            item["question"].lower() + " " +
            item["answer"].lower()
        )

```

```
score = 0

# Count matching words
for word in message.split():

    if word in text:
        score += 1

# Save best match
if score > best_score:
    best_score = score
    best_match = item

# Return answer
if best_match and best_score > 0:
    return best_match["answer"]

return "Sorry, I don't know the answer yet."

# Gradio Chat Interface
demo = gr.ChatInterface(
    fn=chatbot,
    title="RCOEM AI ChatBot",
    description="Ask questions related to RCOEM",
    chatbot=gr.Chatbot(height=500)
)

# Launch App with Public Link
demo.launch(
    share=True,
    debug=True
)
```

/tmp/ipykernel_1419/117737718.py:48: UserWarning: You have not specified a value for the `type` parameter. Defaulting to the chatbot=gr.Chatbot(height=500)
/tmp/ipykernel_1419/117737718.py:48: DeprecationWarning: The default value of 'allow_tags' in gr.Chatbot will be changed from chatbot=gr.Chatbot(height=500)
/usr/local/lib/python3.12/dist-packages/gradio/chat_interface.py:330: UserWarning: The gr.ChatInterface was not provided with warnings.warn(
Colab notebook detected. This cell will run indefinitely so that you can see errors and logs. To turn off, set debug=False in * Running on public URL: <https://5ed1c4e6f01badb8da.gradio.live>

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the

RCOEM AI ChatBot

Ask questions related to RCOEM

 Chatbot

Next steps:  [Deploy to Cloud Run](#)

Start coding or [generate](#) with AI.

